

Domain-Invariant Decoding for Speech Brain-Computer Interfaces

Co-authored with Yihe Xu and Injun Choi — final project for UCLA's ECE C143A/C243A

What We Built

Speech brain-computer interfaces decode intended speech directly from neural recordings, but performance degrades across recording sessions because the underlying signal drifts from day to day. We took a baseline GRU decoder from the Brain-to-Text '24 benchmark and improved it along three independent axes: data augmentation (channel dropout and temporal masking to force the model to rely on distributed, context-based features rather than memorizing individual electrode patterns), domain adversarial training (a gradient reversal layer that penalizes the model for encoding which day a recording came from, forcing day-invariant representations), and a hybrid optimization schedule (SWATS, which switches from Adam to SGD partway through training to escape the sharp minima Adam tends to converge to).

How We Measured It

We ran a controlled ablation study, adding each contribution independently before combining all three, and scored every configuration by phoneme error rate (PER) on held-out validation data.

Configuration	PER	Relative Improvement
Baseline (GRU + Adam)	23.61%	—
Data augmentation alone	21.77%	7.8%
Domain adversarial training alone	22.03%	6.7%
SWATS optimization alone	19.61%	17.0%
All three combined	17.40%	26.3%

The combined system also reached the baseline's converged performance 1.8× faster during training (15.3 minutes versus 28.1 minutes). The ablation was the point of the exercise as much as the final number: it let us attribute the improvement to specific causes instead of reporting one aggregate result and guessing at why it worked. Isolating each piece also showed that SWATS, not the domain adversarial training, drove most of the gain, which ran against our initial assumption that explicitly correcting for day-to-day drift would matter most.

How We Approached the Work

We started from a working baseline rather than building from scratch, then treated each proposed improvement as a hypothesis to be tested in isolation before trusting it in combination. For the domain adversarial piece, we implemented the gradient reversal layer as a custom PyTorch autograd function: it passes data through unchanged on the forward pass, then negates and scales the gradient on the backward pass, so the encoder is penalized for making the recording day easy to identify.

```
class GradReverse(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, lambd):
        ctx.lambd = lambd
        return x.view_as(x)

    @staticmethod
    def backward(ctx, grad_output):
        return -ctx.lambd * grad_output, None
```

For the optimizer switch, the harder problem wasn't deciding to move from Adam to SGD, it was choosing an SGD learning rate that wouldn't destabilize training at the handoff. We addressed that by projecting Adam's per-parameter adaptive learning rates onto a single scalar at the moment of the switch, using Adam's own internal state rather than a manually tuned value:

```
def get_smart_sgd_lr(adam_optimizer, beta2=0.999):
    # Projects Adam's adaptive per-parameter learning rates
    # onto a single scalar SGD learning rate at the switch point.
    ...
    v_hat = v / bias_correction
    layer_effective_lr = base_lr / (torch.sqrt(v_hat) + eps)
    effective_lrs.append(layer_effective_lr.mean().item())
    return np.mean(effective_lrs)
```